Customer Journey Path Optimization

Progress Report

by

Aye Khin Khin Hpone (Yolanda Lim) | Yosakorn Sirisoot

125970 | 126512

AT70.02 – Algorithms Design and Analysis

Instructor: Dr. Chantri Polprasert

Teaching Assistant: Puskar Adhikari

Asian Institute of Technology

School of Engineering and Technology

Thailand

March 2026

ABSTRACT

This project addresses the problem of identifying optimal conversion paths in large-scale customer journey graphs. User interaction sequences are modelled as a directed weighted graph $G = (V, E)$, where edge weights encode the negative log-probability of each state transition, enabling maximum-probability path search to be solved as a shortest-path problem using Dijkstra's algorithm.

Two algorithms were implemented and evaluated. The **Baseline Dijkstra** algorithm provides globally optimal solutions in $O(|E| \log |V|)$ time. The **Probability-Pruned Dijkstra** extends the baseline with a threshold parameter τ that discards low-probability partial paths during traversal, reducing the effective search space to $O(|E'| \log |V|)$ with $|E'| \leq |E|$ while converging to the baseline as $\tau \to 0$.

Both algorithms were evaluated on synthetic graphs up to 10,000 nodes (2,160 experiment rows) across varying edge densities and probability distributions, and validated on two real-world e-commerce clickstream datasets: RetailRocket (2.7M events) and RecSys 2015 YOOCHOOSE (33M clicks), totalling 300 real-data experiment rows. Formal correctness proofs, theoretical complexity analysis, and comprehensive empirical evaluation were completed. All 40 unit tests pass.

Results demonstrate that the pruned algorithm achieves median edge-count speedups ranging from 8.8× ($\tau = 0.001$) to 2,124× ($\tau = 0.5$), with wall-clock speedups reaching 881×. Crucially, the optimality gap is exactly 0.0000% across all 171 path-found rows—the pruned algorithm either returns the exact same optimal path as the baseline or prunes it entirely, confirming the "all-or-nothing" property. On real-world data, wall-clock speedups reach 27,693× (RetailRocket item-level) and 4,191× (RecSys 2015).

CONTENTS

ABSTRACT **ii**

1 INTRODUCTION **1**

1.1 Background of the Study 1

1.2 Statement of the Problem 1

1.3 Objectives 2

1.4 Hypothesis and Verdict 3

1.5 Target Users 3

2 RELATED WORK AND BACKGROUND **4**

2.1 Shortest-Path Algorithms for Conversion Path Optimization 4

2.2 Heuristic and Approximate Shortest-Path Approaches 4

2.3 Markov Chain Models for Probabilistic Navigation 5

2.4 Algorithmic Foundations of Efficient Graph Traversal 5

2.5 Sequential Pattern Mining for Frequent Journey Discovery 6

2.6 Survey of Frequent and Sequential Pattern Mining Techniques 6

2.7 Modeling User Navigation as Directed Graphs 6

2.8 Probabilistic Models for Navigation and Path Selection 7

2.9 Scalable Graph Processing and Large-Scale Navigation Analysis 7

2.10 Real-World E-Commerce Clickstream Datasets 8

2.11 Graph Cycles in Customer Journey Navigation 9

2.12 Synthetic Graph Generation for Controlled Evaluation 9

3 ALGORITHM DESIGN, DATA STRUCTURES, AND IMPLEMENTATION **11**

3.1 System Overview 11

3.2 Problem Formulation 12
3.2.1 Graph Model and Inputs 12
3.2.2 Probabilistic Path Objective and Outputs 12
3.2.3 Constraints and Assumptions 13
3.3 Data Generation and Preprocessing 13
3.3.1 Synthetic Data Generation 13
3.3.2 Real-World Clickstream Data 14
3.3.3 Preprocessing and Input Formats 15
3.4 Graph Construction 15
3.5 Algorithmic Approaches 16
3.5.1 Baseline Algorithm: Dijkstra's Shortest Path 16
3.5.2 Optimized Algorithm: Probability-Pruned Dijkstra 17
3.5.3 Algorithm Comparison 20
3.5.4 Critical-τ Finder 20
3.5.5 k-Shortest Simple Paths 21
3.6 Data Structures 21
3.6.1 Adjacency List 21
3.6.2 Binary Min-Heap (Priority Queue) 21
3.6.3 Hash Maps (Dictionaries) 22
3.7 Experimental Design 22
3.7.1 Simulation Environment 22
3.7.2 Evaluation Metrics 22
3.7.3 Experimental Parameters 23
3.7.4 Experimental Controls 23

4 CORRECTNESS PROOFS AND VALIDATION 24

4.1 Correctness of the Log-Probability Transformation 24
4.2 Correctness of Baseline Dijkstra 24

4.3 Correctness of Probability-Pruned Dijkstra 25
4.3.1 Convergence at $\tau = 0$ 25
4.3.2 Conditional Optimality for $\tau > 0$ 25
4.3.3 All-or-Nothing Property 26
4.4 Empirical Validation Results 26

5 COMPLEXITY ANALYSIS AND EMPIRICAL VALIDATION 28

5.1 Analytical Methodology 28
5.2 Time Complexity 28
5.2.1 Baseline Dijkstra 28
5.2.2 Probability-Pruned Dijkstra 29
5.2.3 Empirical Time Validation 29
5.3 Space Complexity 29
5.3.1 Empirical Memory Validation 30
5.4 Summary of Complexity 30

6 EXPERIMENTAL RESULTS AND ANALYSIS 31

6.1 Synthetic Experiment Results 31
6.1.1 Speedup vs. Pruning Threshold τ 31
6.1.2 Speedup vs. Graph Size $|V|$ 31
6.1.3 Speedup by Graph Type 32
6.1.4 Speedup by Probability Distribution 33
6.1.5 Optimality Gap Analysis 34
6.2 Real-Data Experiment Results 34
6.2.1 Datasets and Graph Statistics 34
6.2.2 Real-Data Speedup Results 34
6.2.3 Reachability Analysis 35
6.3 Summary of Findings 35

7 CONCLUSION AND FUTURE WORK **37**

7.1 Summary of Contributions 37

7.2 Limitations 38

7.3 Future Work 38

8 PROJECT PLAN **39**

8.1 Project Timeline 39

8.2 Milestones 39

REFERENCES **41**

APPENDIX A: SOURCE CODE AVAILABILITY **43**

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Understanding how users navigate through digital systems—such as e-commerce websites, mobile applications, and customer support platforms—is essential for improving conversion rates, user satisfaction, customer retention, and operational efficiency. User interaction data captures detailed behavioural patterns that reflect how users interact with different system components. Customer journeys, represented as sequences of interaction states (e.g., *Landing* $\rightarrow$ *Browse* $\rightarrow$ *Product* $\rightarrow$ *Cart* $\rightarrow$ *Checkout*), provide a structured framework for analyzing these behaviours.

Modern digital systems generate millions of user sessions and thousands of interaction states, forming large and complex navigation graphs across websites and online platforms. When aggregated, these journeys form large directed graphs with numerous states and transitions. The scale and complexity of such graphs make efficient computational analysis a necessity, motivating the use of algorithmic approaches grounded in graph theory and data mining.

1.2 Statement of the Problem

As the volume and complexity of user interaction data continue to grow, identifying meaningful and optimal customer journeys becomes increasingly challenging. From a business perspective, customer journey dynamics directly influence conversion rates, revenue generation, customer retention, and user satisfaction. Even small improvements in the probability of reaching a conversion state can result in significant financial gains at scale.

Naïve analytical methods, such as exhaustive path enumeration or simple frequency-based techniques, do not scale. In large graphs, the number of possible simple paths increases combinatorially, rendering exhaustive enumeration computationally infeasible. Furthermore, directly multiplying many small transition probabilities along a path leads to numerical underflow—a form of arithmetic precision loss that makes naïve probability computation unreliable for longer paths.

The key challenge is therefore to identify optimal customer journeys efficiently and numerically

stably at scale. Core tasks include finding highest-probability conversion paths and analyzing scalability across large graphs, each posing distinct challenges in graph traversal, optimization, and algorithm design that require careful data structure selection.

1.3 Objectives

The objective of this study is to design and implement an efficient algorithmic framework for customer journey path optimization grounded in graph theory and classical algorithm design principles. Specifically, the project models customer journey data as a directed weighted graph, where nodes represent interaction states and edges represent transition probabilities between states.

The primary focus is to compute optimal conversion paths under probabilistic constraints. To achieve this, transition probabilities are transformed into additive non-negative edge weights using the negative logarithm, enabling the application of Dijkstra's shortest-path algorithm for maximum-probability path computation. This transformation also addresses the numerical underflow problem by converting multiplicative probability products into additive sums. The project implements Dijkstra's algorithm as a baseline approach and proposes an optimized probability-pruned search strategy that reduces the exploration of low-probability paths to improve computational efficiency.

In addition to correctness, the project formally analyzes the theoretical time and space complexity of both the baseline and optimized algorithms. Empirical performance evaluations have been conducted on graphs of varying sizes and densities to examine scalability, runtime behaviour, and trade-offs between computational efficiency and path optimality. Through these analyses, the project demonstrates how careful algorithm design and data structure selection influence performance in large-scale navigation graphs.

Original contribution. This study contributes a threshold-based pruning strategy integrated with Dijkstra's algorithm for probabilistic navigation graphs, evaluated through empirical complexity analysis and an explicit time–optimality trade-off characterization. Unlike heuristic approaches (A*) that require a problem-specific admissible heuristic, or rank-based methods (beam search) that provide no convergence guarantee, the proposed pruning is deterministic, requires no domain-specific heuristic, and formally converges to the globally optimal solution as $\tau \to 0$.

A further contribution is the identification of the *critical pruning threshold* τ_c, defined as the value or narrow range of τ for which runtime improvement is maximized without causing any degradation in path quality. Empirical results confirm that the pruning mechanism is "all-or-nothing"—it either preserves the exact optimal path (0% gap) or prunes it entirely.

1.4 Hypothesis and Verdict

Hypothesis:

> Probability-Pruned Dijkstra provides meaningful speedup over baseline Dijkstra while preserving path optimality at conservative τ values.

Experimental Verdict: Hypothesis Supported.

Pruning reduces the number of edges examined, yielding speedups from $8.8\times$ at $\tau = 0.001$ to $2{,}124\times$ at $\tau = 0.5$. Whenever the pruned algorithm finds a path, it returns the exact same optimal path as the baseline, with an optimality gap of 0.0000% across all 171 path-found rows. The only cost is reduced reachability: at aggressive τ values the optimal path may be pruned entirely, causing the algorithm to report no path.

1.5 Target Users

The primary target users of this system include data analysts, product managers, and system designers working on digital platforms such as e-commerce websites, mobile applications, and customer support systems. The project is also intended for researchers and students interested in applying algorithm design and data structure concepts to real-world optimization problems. Insights derived from this work can support data-driven decision-making and contribute to improved system performance and user experience.

CHAPTER 2

RELATED WORK AND BACKGROUND

2.1 Shortest-Path Algorithms for Conversion Path Optimization

Dijkstra [Dijkstra, 1959] introduced one of the most fundamental shortest-path algorithms for weighted graphs with non-negative edge weights. The algorithm incrementally explores nodes in order of increasing accumulated cost using a priority queue, guaranteeing optimality when all edge weights are non-negative. When implemented with a binary min-heap, it achieves a time complexity of $O(|E| \log |V|)$ [Cormen et al., 2009]. A theoretically superior bound of $O(|E| + |V| \log |V|)$ is achievable using Fibonacci heaps [Fredman and Tarjan, 1987], though the practical overhead of Fibonacci heaps often makes binary-heap implementations preferable for moderately sized graphs. In the context of this project, Dijkstra's algorithm serves as the baseline, applied to a log-transformed probability graph where edge weights represent the negative log-probability of transitions. Its well-understood complexity and correctness guarantees make it the natural choice for baseline comparison and scalability analysis.

2.2 Heuristic and Approximate Shortest-Path Approaches

While Dijkstra's algorithm guarantees optimality, it explores the full reachable graph from the source. Informed search algorithms such as A* [Hart et al., 1968] reduce exploration by using an admissible heuristic function to guide node expansion toward the target, achieving sub-Dijkstra runtime when a valid heuristic exists. Pearl [Pearl, 1984] provides a comprehensive theoretical treatment of heuristic search, establishing formal conditions under which informed search algorithms guarantee optimal solutions. In probabilistic graph settings, however, designing a tight and admissible heuristic over log-probability weights is non-trivial, making heuristic approaches difficult to apply directly. Beam search offers an alternative approximation strategy by limiting the frontier to the top-k most promising candidates at each step [Cormen et al., 2009]. Although beam search can be very fast in practice, it provides no optimality guarantee and may miss the globally optimal path entirely. This project adopts a different approximation strategy—probability-threshold pruning—which is deterministic, parameter-controlled, and guarantees convergence to the optimal solution as the threshold approaches zero, offering

a more principled trade-off than beam search. Unlike A*, which requires a problem-specific admissible heuristic that is difficult to define over log-probability edge weights, and unlike beam search, which discards nodes based on rank with no convergence guarantee, the proposed pruning operates directly on accumulated path cost against a single interpretable parameter τ, requiring no domain-specific heuristic design and providing a formal convergence proof as $\tau \to 0$.

Another important family of shortest-path speedup methods is *Contraction Hierarchies*, which accelerates queries by preprocessing the graph and adding shortcut edges to preserve shortest-path distances. Such methods are highly effective for static road-network-style graphs, but they are less naturally suited to the probabilistic customer-journey setting considered here, where transition weights are derived from behavioral data and experimental emphasis is placed on controllable approximation rather than heavy preprocessing.

2.3 Markov Chain Models for Probabilistic Navigation

The probabilistic transition model used in this project—where each user state transitions to the next with a fixed probability—is formally equivalent to a first-order Markov chain [Norris, 1998]. Markov chains have been widely used to model sequential user behaviour in web navigation, recommendation systems, and clickstream analysis [Rabiner, 1989, Baum and Petrie, 1966]. The maximum-probability path problem on a Markov chain is structurally identical to the Viterbi algorithm [Viterbi, 1967], which finds the most likely state sequence in a Hidden Markov Model using dynamic programming in $O(|V|^2 T)$ time for T time steps. However, the Viterbi algorithm is designed for sequences of fixed length, whereas the present problem involves finding the most probable *path* from s to t in a general directed graph without a fixed path length. The log-probability shortest-path formulation adopted here generalizes the Viterbi principle to arbitrary directed graphs and applies it efficiently using Dijkstra's algorithm, which has better practical complexity on sparse graphs.

2.4 Algorithmic Foundations of Efficient Graph Traversal

Cormen et al. [Cormen et al., 2009] provided a comprehensive treatment of graph algorithms, including shortest-path search, graph traversal, and priority-queue-based optimization techniques. The textbook formally analyzes time and space complexity, establishes correctness proofs for Dijkstra's algorithm, and discusses data structure choices including binary heaps

and adjacency lists. These foundations directly support the design decisions in this project: the use of adjacency lists for $O(|V|+|E|)$ space, binary heaps for $O(\log|V|)$ extract-min and insert operations, and hash maps for $O(1)$ distance lookup. The log-probability weight transformation exploits the monotone relationship between probability maximization and shortest-path minimization, a standard technique in probabilistic graph optimization [Manning and Schütze, 1999].

2.5 Sequential Pattern Mining for Frequent Journey Discovery

Pei et al. [Pei et al., 2001] proposed PrefixSpan, a pattern-growth-based algorithm for mining frequent sequential patterns without candidate generation. By projecting the database based on frequent prefixes, PrefixSpan significantly reduces search space compared to Apriori-based approaches and demonstrated strong scalability on large sequence datasets. In the context of this project, PrefixSpan is conceptually relevant as a method for identifying frequently taken customer journey paths without enumerating all possible sequences. Its prefix-based pruning principle shares conceptual similarities with the threshold-based pruning strategy proposed in this work, where low-probability path prefixes are discarded early to reduce the effective search space.

2.6 Survey of Frequent and Sequential Pattern Mining Techniques

Han et al. [Han et al., 2007] presented a comprehensive survey of frequent pattern mining methods, covering association rules, sequential patterns, and constrained pattern mining. The survey highlights key challenges including scalability, pattern explosion, and the need for efficient pruning strategies. These challenges closely mirror those encountered in customer journey analysis, where the number of simple paths from source to target grows combinatorially with graph size. This project draws on the general principle of threshold-based pruning to control computational complexity, filtering out low-probability paths before full expansion in a manner analogous to minimum support thresholds in frequent pattern mining.

2.7 Modeling User Navigation as Directed Graphs

Bucklin and Sismeiro [Bucklin and Sismeiro, 2003] modeled user browsing behaviour using clickstream data, representing navigation patterns as structured sequences and probabilistic transition graphs. Their empirical analysis demonstrated that user navigation can be effectively

captured using probabilistic transition models, providing insights into user behaviour and engagement at scale. Liu [Liu, 2011] further surveyed web usage mining techniques—including clickstream analysis, hyperlink analysis, and content mining—providing a broader context for how navigation data is processed into actionable graph representations. This work directly supports the modeling approach adopted in this project, where customer journeys are represented as directed weighted graphs derived from interaction sequences, and transition probabilities are estimated from session frequency counts using maximum likelihood estimation.

2.8 Probabilistic Models for Navigation and Path Selection

Resnick and Varian [Resnick and Varian, 1997] discussed probabilistic models of user behaviour, including transition-based navigation models that capture the likelihoods of moving between system states. Such models are commonly used to estimate user preferences and navigation patterns in large-scale recommendation and information retrieval systems. The probabilistic transition modeling approach adopted in this project—assigning edge weights as negative log-probabilities and finding the minimum-weight path—is consistent with the broader class of log-linear models used in language modeling and information retrieval [Manning and Schütze, 1999, Manning et al., 2008], where the negative log-likelihood serves as a natural additive cost function.

2.9 Scalable Graph Processing and Large-Scale Navigation Analysis

Recent advances in large-scale graph processing systems further emphasize the importance of efficient algorithmic design when analyzing massive navigation graphs. Malewicz et al. [Malewicz et al., 2010] introduced Pregel, a vertex-centric computational model for scalable graph processing, demonstrating how distributed systems can efficiently handle graphs containing millions or billions of edges. Subsequent research in large-scale graph analytics has examined optimization strategies for traversal, pruning, and distributed processing to reduce computational overhead in dense and evolving networks [Sahu et al., 2017]. These studies reinforce the practical relevance of designing efficient path-search algorithms and pruning strategies when working with large customer journey graphs, where computational complexity grows rapidly with graph size. The present project builds upon these principles by evaluating algorithmic efficiency and search-space reduction techniques within a single-machine, controlled experimental framework, where the goal is to characterize complexity empirically rather than to achieve dis-

tributed scalability.

2.10 Real-World E-Commerce Clickstream Datasets

Two public e-commerce clickstream datasets were used for real-world validation in this project:

RetailRocket E-Commerce Dataset [Retail Rocket, 2015] contains approximately 2.7 million clickstream events across 1.4 million visitors, collected from a real e-commerce platform. It provides three event types (`view`, `addtocart`, `transaction`) with item-level granularity, enabling graph construction at two scales: a small 3-node event-type funnel and a large ∼44,700-node item-level graph.

RecSys Challenge 2015 (YOOCHOOSE) [Ben-Shimon et al., 2015] contains approximately 33 million click events across 9.2 million sessions from an online retailer. Item-to-item click transitions within sessions form a ∼12,900-node, ∼70,400-edge graph when restricted to 50,000 sessions. This dataset provides dense clickstream data suitable for testing algorithmic scalability on real-world transition patterns.

These datasets replaced the UCI E-Commerce Clickstream dataset originally planned in the proposal, as YOOCHOOSE provides substantially larger scale and more representative e-commerce behaviour patterns for validation purposes.

The reviewed literature establishes a strong theoretical and methodological foundation for this project. Dijkstra's algorithm [Dijkstra, 1959] and its complexity analysis [Cormen et al., 2009, Fredman and Tarjan, 1987] provide the basis for the baseline implementation and complexity comparison. The Markov chain and Viterbi frameworks [Norris, 1998, Viterbi, 1967, Rabiner, 1989] situate the probabilistic path problem in a well-studied theoretical context. Heuristic search methods [Hart et al., 1968, Pearl, 1984] and approximate approaches establish the landscape of alternatives against which the pruning strategy is positioned. Sequential pattern mining techniques [Pei et al., 2001, Han et al., 2007] and clickstream modeling [Bucklin and Sismeiro, 2003, Liu, 2011] validate the representation and motivate pruning as a scalability mechanism. Probabilistic modeling literature [Resnick and Varian, 1997, Manning and Schütze, 1999] supports the log-probability transformation. Scalable graph systems [Malewicz et al., 2010, Sahu et al., 2017] contextualize the scalability challenges addressed. The Erdős–Rényi model [Erdős and Rényi, 1960] grounds the synthetic data generation methodology, and real-world benchmarks [Retail Rocket, 2015, Ben-Shimon et al., 2015] validate experimental generalizability.

2.11 Graph Cycles in Customer Journey Navigation

A critical modeling question is whether the customer journey graph contains cycles—that is, whether a user may return to a previously visited state, such as *Landing* $\rightarrow$ *Browse* $\rightarrow$ *Product* $\rightarrow$ *Browse*. Real-world clickstream data frequently exhibits such revisitation behaviour [Bucklin and Sismeiro, 2003]: users loop between states (e.g., browsing multiple product pages before adding to cart) before eventually converting or abandoning the session. The customer journey graph is therefore modeled as a general directed weighted graph that *may contain cycles*.

A key concern is whether Dijkstra's algorithm remains correct on cyclic graphs. Cormen et al. [Cormen et al., 2009] establish that Dijkstra's algorithm is correct on any directed graph with non-negative edge weights, regardless of whether cycles are present. The log-probability transformation $w(u,v) = -\log p(u,v) \geq 0$ guarantees non-negativity for all $p(u,v) \in (0,1]$, so Dijkstra's algorithm is directly applicable to cyclic navigation graphs without any cycle-detection preprocessing. Since each cycle traversal adds strictly positive cost, the algorithm naturally avoids cycling: once a node is finalized, it is never re-extracted from the priority queue. This property means no modifications to the standard algorithm are required to handle the cyclic structure that is inherent in real user navigation data.

2.12 Synthetic Graph Generation for Controlled Evaluation

To evaluate algorithmic performance in a controlled and reproducible manner, synthetic graphs are used as the primary experimental dataset. The generation methodology is grounded in the Erdős–Rényi random graph model [Erdős and Rényi, 1960], which provides a theoretically principled basis for generating random directed graphs with predictable structural properties (connectivity, degree distribution) at a given size and density.

Two graph generators were implemented. The first, an **Erdős–Rényi sparse random generator**, samples directed edges independently with expected total $\bar{d} \cdot |V|$ edges and assigns transition probabilities under uniform $\mathscr{U}(0.01, 1.0)$ or skewed power-law ($\alpha = 2$) distributions. The second, a **layered (stage-based) generator**, partitions nodes into five ordered stages (Awareness, Interest, Consideration, Intent, Conversion) with edges going primarily forward (up to $+2$ stages) and a configurable `backward_prob` parameter controlling backward links modelling user loops.

Both generators use $O(n \cdot d)$ scalable edge sampling, guarantee $s \to t$ connectivity via BFS + bridge edge, normalize outgoing probabilities to sum to 1, and use deterministic seeds via `hashlib.md5` for cross-platform reproducibility.

Research Gap. Despite the extensive literature above, existing studies largely treat shortest-path computation [Dijkstra, 1959, Cormen et al., 2009], probabilistic modeling [Norris, 1998], and sequential pattern mining [Pei et al., 2001] as separate problems. Limited work integrates all three within a unified framework tailored to customer conversion path analysis, with explicit treatment of cyclic navigation graphs, principled synthetic data generation [Erdős and Rényi, 1960], and empirical characterization of the time–optimality trade-off of threshold-based pruning.

The present project addresses this gap by combining probabilistic transition modeling, threshold-controlled pruning, and Dijkstra's algorithm into a cohesive experimental framework. By systematically evaluating performance across diverse synthetic and real-world graph configurations—including cyclic graphs—the study bridges classical graph algorithms, Markov chain theory, and practical navigation graph analysis within a single experimental design.

CHAPTER 3

ALGORITHM DESIGN, DATA STRUCTURES, AND IMPLEMENTATION

3.1 System Overview

The *Customer Journey Path Optimization* system models user interactions as a directed weighted graph $G = (V, E)$, where each node $v \in V$ represents a discrete interaction state (e.g., *Landing Page*, *Search*, *Product Page*, *Cart*, *Checkout*), and each directed edge $(u, v) \in E$ represents a user transition between states annotated with a transition probability $p(u, v) \in (0, 1]$. The primary goal is to identify the most probable path from a designated source node s to a target conversion node t.

Figure 3.1 presents the end-to-end workflow process. The workflow process is organized into five sequential components: (1) data generation and preprocessing, (2) graph construction, (3) baseline path optimization using Dijkstra's algorithm, (4) optimized path search using probability-pruned Dijkstra, and (5) experimental evaluation and comparative analysis.

Two algorithms were implemented and compared throughout this chapter. Algorithm 1 (Baseline Dijkstra on Log-Weights) applies standard Dijkstra's algorithm to the log-probability-transformed graph, guaranteeing globally optimal paths in $O(|E| \log |V|)$ time. Algorithm 2 (Probability-Pruned Dijkstra) extends Algorithm 1 with a pruning threshold τ that discards partial paths whose cumulative probability falls below τ, reducing expected runtime to $O(|E'| \log |V|)$, $|E'| \leq |E|$, with convergence to Algorithm 1 as $\tau \to 0$.

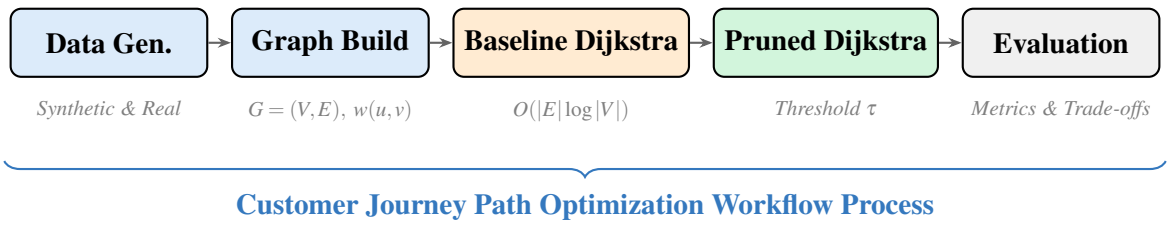

Figure 3.1. Five-stage workflow process: data generation, graph construction, baseline Dijkstra, pruned Dijkstra, and experimental evaluation.

3.2 Problem Formulation

3.2.1 Graph Model and Inputs

Customer interactions are modeled as a directed graph $G = (V, E)$, where V is the set of interaction states (nodes), $E \subseteq V \times V$ is the set of directed transitions (edges), and each edge $(u, v) \in E$ carries a transition probability $p(u, v) \in (0, 1]$. A designated source node $s \in V$ (e.g., Landing Page) and target node $t \in V$ (e.g., Checkout) define the query endpoints. In the optimized algorithm, a pruning threshold $\tau \in (0, 1]$ is used to discard low-probability partial paths.

Figure 3.2 illustrates a representative small customer journey graph to clarify the graph model.

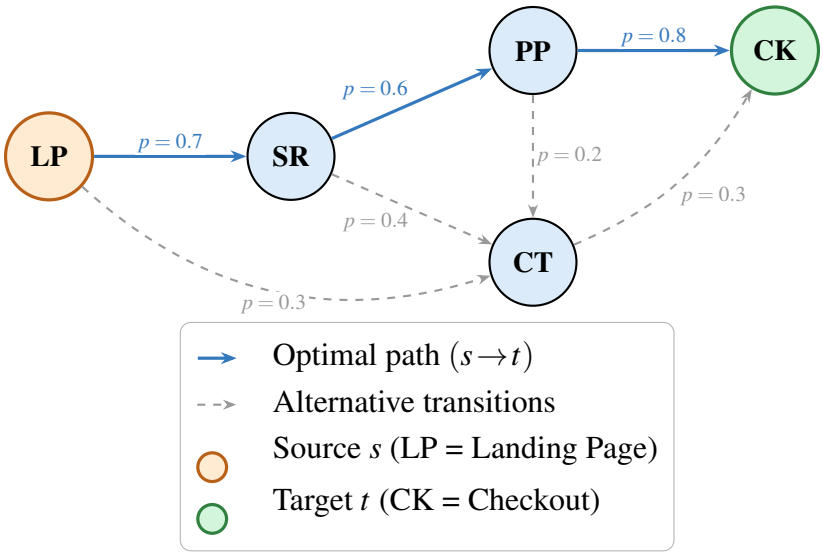

Figure 3.2. Representative customer journey graph. LP = source s, CK = target t. Solid blue path LP→SR→PP→CK is optimal ($\Pi^* = 0.336$, log-cost≈ 1.091).

3.2.2 Probabilistic Path Objective and Outputs

The goal is to find a path $P^* = (v_0 = s, v_1, \ldots, v_k = t)$ that maximizes the joint path probability [Norris, 1998]:

$$\max_{P} \prod_{(u,v) \in P} p(u, v)$$

Directly multiplying many small probabilities causes numerical underflow [Manning and Schütze, 1999]. Each transition probability is transformed into an additive non-negative edge weight using the negative logarithm:

$$w(u,v) = -\log\big(p(u,v)\big)$$

Since $p(u,v) \in (0,1]$, it follows that $w(u,v) \geq 0$, satisfying Dijkstra's non-negativity requirement [Cormen et al., 2009]:

$$\max \prod_{(u,v)\in P^*} p(u,v) \iff \min \sum_{(u,v)\in P^*} -\log p(u,v)$$

For a given source–target query, the system returns the optimal path $P^* = (v_0 = s, v_1, \ldots, v_k = t)$, the corresponding path probability $\Pi^* = \prod_{(u,v)\in P^*} p(u,v) = \exp(-C^*)$, and the log-cost $C^* = \sum_{(u,v)\in P^*} -\log p(u,v)$. Each run also records performance metrics—execution time, peak memory, nodes explored, edges relaxed, and maximum priority queue size—as well as the optimality gap $\Delta = |C^*_{\text{baseline}} - C^*_{\text{pruned}}|/C^*_{\text{baseline}} \times 100\%$.

3.2.3 Constraints and Assumptions

The framework assumes graph sizes up to 50,000 nodes (hardware permitting), with experiments conducted on graphs of up to 10,000 nodes. Graphs are assumed to be sparse ($|E| = O(|V|)$) and stored using an adjacency-list representation requiring $O(|V| + |E|)$ space. All transition probabilities are strictly positive; zero-probability transitions are treated as non-existent edges. The customer journey graph may contain directed cycles; correctness is preserved because $w(u,v) \geq 0$ for all edges. Finally, a first-order **Markov assumption** is adopted: $P(\text{next} \mid \text{current})$ is independent of earlier states.

3.3 Data Generation and Preprocessing

3.3.1 Synthetic Data Generation

Two graph generators were implemented in `data/graph_generator.py`. The first is an **Erdős–Rényi sparse random generator** [Erdős and Rényi, 1960], which creates $|V|$ nodes and samples directed edges with expected total $\bar{d} \cdot |V|$; transition probabilities are drawn from either a uniform $\mathscr{U}(0.01, 1.0)$ or a power-law ($\alpha = 2$) distribution. The second is a **layered (stage-based) generator** in which nodes are distributed across five ordered stages (Awareness, Interest, Consideration, Intent, Conversion), edges proceed primarily forward (up to $+2$ stages), and a configurable `backward_prob` parameter controls backward links.

Both generators employ $O(n \cdot d)$ scalable edge sampling rather than $O(n^2)$ adjacency-matrix construction. A BFS-based connectivity guarantee ensures that if no $s \to t$ path exists after random sampling, bridge edges are added. Outgoing probabilities are normalised per node ($\sum_v p(u,v) = 1$), and deterministic seeding via `hashlib.md5` ensures cross-platform reproducibility.

The following code snippet shows key aspects of the Erdős–Rényi generator:

```
def generate_erdos_renyi_graph(n, avg_degree, distribution, seed,
    ...):
    rng = random.Random(seed)
    graph = {f"node_{i}": {} for i in range(n)}
    # O(n*d) edge sampling -- NOT O(n^2)
    for u in graph:
        num_edges = rng.randint(
            max(0, int(avg_degree) - 2), int(avg_degree) + 2)
        targets = rng.sample(other_nodes, min(num_edges, len(
            other_nodes)))
        for v in targets:
            raw = rng.random() if distribution == "uniform" \
                  else rng.paretovariate(2)
            graph[u][v] = raw
    _normalize_and_log_transform(graph)
    _ensure_connectivity(graph, source, target)
    return graph
```

Listing 3.1: Core edge sampling from the ER generator (`data/graph_generator.py`)

3.3.2 Real-World Clickstream Data

Two real-world clickstream datasets were used for validation (Table 3.1):

Dataset	**Events**	**Sessions**	**Graph Size**	**Source**
RetailRocket (event-level)	2.7M	1.4M	3 nodes, 9 edges	Kaggle [Retail Rocket, 2015]
RetailRocket (item-level)	2.7M	50K	44,711 nodes, 101,528 edges	Kaggle [Retail Rocket, 2015]
RecSys 2015 (YOOCHOOSE)	33M	50K	12,935 nodes, 70,442 edges	Kaggle [Ben-Shimon et al., 2015]

Table 3.1. Real-world clickstream datasets used for validation. RetailRocket supports two granularities: event-type (3-node funnel) and item-level (large graph). RecSys 2015 uses item-to-item click transitions.

Transition probabilities are estimated using maximum likelihood:

$$\hat{p}(u,v) = \frac{\text{count}(u \to v)}{\sum_{w:(u,w)\in E} \text{count}(u \to w)}$$

3.3.3 Preprocessing and Input Formats

The preprocessing module (`src/preprocessing.py`) accepts two CSV formats:

1. **Direct transitions** — columns `source` and `target`, one row per observed transition.
2. **Session event streams** — columns `session_id` and `state` (plus `step` or `timestamp` for ordering). Consecutive events within each session are converted to pairwise transitions.

Both formats produce the same transition-count table from which MLE probabilities are computed.

3.4 Graph Construction

The directed weighted graph is constructed from preprocessed session data using an **adjacency list** representation. Each node u maps to a list of $(v,\ w(u,v))$ pairs, where $w(u,v) = -\log(\hat{p}(u,v))$. This provides $O(|V|+|E|)$ space complexity and efficient $O(\deg(u))$ neighbour iteration [Cormen et al., 2009].

```
def build_weighted_graph(transitions_df, stats_df):
    graph = {}
    for _, row in stats_df.iterrows():
        src, tgt = row["source"], row["target"]
        prob = row["probability"]
        weight = -math.log(prob)  # log-probability transform
        graph.setdefault(src, {})[tgt] = weight
        graph.setdefault(tgt, {})  # ensure sink nodes exist
    return graph
```

Listing 3.2: Graph construction (`src/graph_builder.py`)

3.5 Algorithmic Approaches

3.5.1 Baseline Algorithm: Dijkstra's Shortest Path

The baseline applies Dijkstra's algorithm [Dijkstra, 1959] to the log-transformed graph. A **binary min-heap** (`heapq`) serves as the priority queue with **lazy insertion**—when a shorter path to a node is found, a new entry is pushed and stale entries are skipped when popped.

Algorithm 1: Dijkstra on Transformed Probabilities [Dijkstra, 1959, Cormen et al., 2009]

Input: Graph $G = (V, E)$ with weights $w(u, v) = -\log p(u, v)$, source s, target t

Output: Shortest-path distance $dist[t]$, probability $\exp(-dist[t])$, path P^*

```
Initialise dist[v] ← ∞ for all v ∈ V; parent[v] ← NIL;
dist[s] ← 0; insert (0, s) into min-heap Q;
while Q is not empty do
    (d, u) ← Extract-Min(Q);
    if u = t then
        break;                                  // optimal path reached
    end
    if d > dist[u] then
        continue;                               // stale entry (lazy deletion)
    end
    foreach edge (u, v) with weight w = −log p(u, v) do
        edges_relaxed ← edges_relaxed + 1;
        new_dist ← dist[u] + w;
        if new_dist < dist[v] then
            dist[v] ← new_dist; parent[v] ← u;
            Insert (new_dist, v) into Q;
        end
    end
end
return dist[t], exp(−dist[t]), ReconstructPath(parent, s, t);
```

Time complexity: $O(|E| \log |V|)$ with a binary heap [Cormen et al., 2009].

Space complexity: $O(|V| + |E|)$.

Optimality: Guaranteed, since all $w(u,v) \geq 0$ [Dijkstra, 1959].

Worked Example

Table 3.2 traces Algorithm 1 on the five-node graph (Figure 3.2).

Step #	**Extract**	**LP**	**SR**	**PP**	**CT**	**CK**
Init	—	0	∞	∞	∞	∞
1	LP	0	0.36	∞	1.20	∞
2	SR	0	0.36	0.87	1.20	∞
3	PP	0	0.36	0.87	1.20	1.09
4	**CK✓**	0	0.36	0.87	1.20	**1.09**

Table 3.2. Dijkstra `dist[]` trace on the five-node graph, $s = \text{LP}$, $t = \text{CK}$. At step 4, CK is finalised with optimal log-cost 1.09, probability $\exp(-1.09) = 0.336$.

3.5.2 Optimized Algorithm: Probability-Pruned Dijkstra

The pruned algorithm extends the baseline by introducing a **probability pruning threshold** $\tau \in (0,1]$. Any partial path whose cumulative probability falls below τ is pruned. In log-space:

$$\text{new_cost} > T = -\log(\tau) \implies \text{skip edge}$$

Algorithm 2: Probability-Pruned Dijkstra

Input: Graph $G = (V, E)$, source s, target t, pruning threshold $\tau \in (0, 1]$

Output: Shortest-path distance $dist[t]$, probability $\exp(-dist[t])$, path P^*

```
Initialise dist[v] ← ∞ for all v ∈ V; parent[v] ← NIL;
dist[s] ← 0; insert (0, s) into min-heap Q;
T ← −log(τ);                                  // pruning threshold in log-space
while Q is not empty do
    (d, u) ← Extract-Min(Q);
    if u = t then
        break
    end
    if d > dist[u] then
        continue;                                          // stale entry
    end
    foreach edge (u, v) with weight w = −log p(u, v) do
        new_dist ← dist[u] + w;
        if new_dist > T then
            continue;                                  // PRUNE: below τ
        end
        edges_relaxed ← edges_relaxed + 1;
        if new_dist < dist[v] then
            dist[v] ← new_dist; parent[v] ← u;
            Insert (new_dist, v) into Q;
        end
    end
end
return dist[t], exp(−dist[t]), ReconstructPath(parent, s, t);
```

Worst-case time complexity: $O(|E| \log |V|)$ (when $\tau \to 0$).

Expected time complexity: $O(|E'| \log |V|)$, $|E'| \leq |E|$.

Space complexity: $O(|V| + |E|)$.

The `edges_relaxed` Metric

The `edges_relaxed` counter has a precise, intentional definition that differs between the two algorithms:

Algorithm	Counts	Placement in Code
Baseline	Every outgoing edge examined from a settled node	After computing `new_cost`, before the improvement check
Pruned	Every edge that **survived** the τ threshold	After the `if new_cost > T: continue` check

Table 3.3. The `edges_relaxed` metric definition. The difference baseline − pruned measures exactly the work saved by pruning.

This design cleanly isolates the pruning effect from stale-entry skips, making the speedup ratio directly attributable to the τ mechanism.

Actual Python Implementation

The following is the actual pruned Dijkstra implementation from the codebase:

```python
def dijkstra_pruned(graph, source, target, tau):
    T = -math.log(tau) if tau > 0 else float("inf")
    dist = {source: 0.0}
    parent = {source: None}
    pq = [(0.0, source)]
    metrics = DijkstraMetrics()

    while pq:
        d, u = heapq.heappop(pq)
        if u == target:
            break
        if d > dist.get(u, float("inf")):
            continue  # stale entry
        metrics.nodes_explored += 1
        metrics.max_pq_size = max(metrics.max_pq_size, len(pq)+1)
        for v, w in graph.get(u, {}).items():
            new_cost = d + w
            if new_cost > T:
                continue  # PRUNE
            metrics.edges_relaxed += 1
            if new_cost < dist.get(v, float("inf")):
                dist[v] = new_cost
                parent[v] = u
                heapq.heappush(pq, (new_cost, v))
```

```
# ... path reconstruction and probability computation
```

Listing 3.3: Pruned Dijkstra implementation (`src/dijkstra.py`)

3.5.3 *Algorithm Comparison*

Figure 3.3 illustrates the decision-flow difference between both algorithms. Table 3.4 summarises the trade-offs.

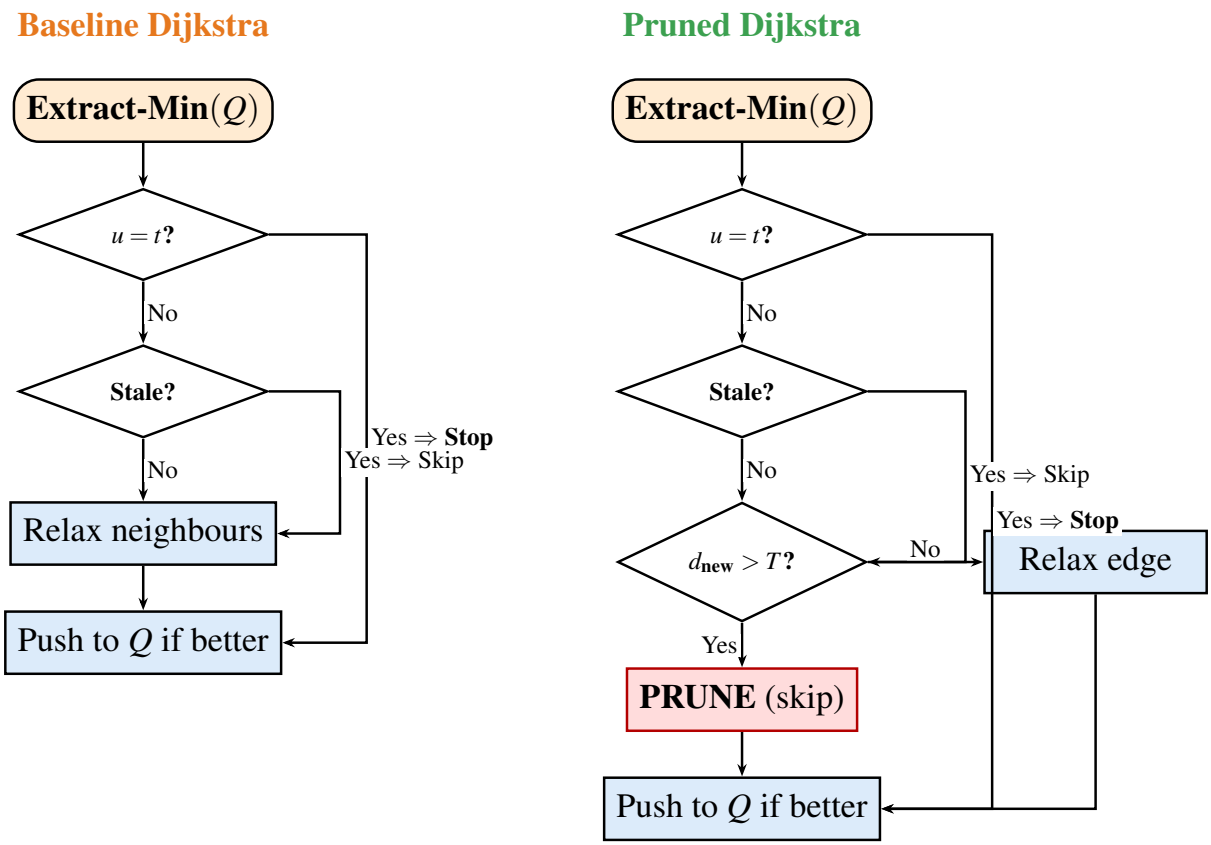

Figure 3.3. Decision-flow comparison. The pruned variant adds a threshold check $d_{\text{new}} > T$ (red box); edges exceeding $T = -\log(\tau)$ are discarded without relaxation.

3.5.4 *Critical-τ Finder*

A practical question is: *what is the largest threshold τ^* that still preserves the optimal path?* The module `src/critical_tau.py` answers this with an adaptive sweep centred on the baseline path probability.

Given the baseline optimal probability Π^*, fixed τ values would miss the interesting region when Π^* is very small (e.g. 10^{-4}). Instead, the sweep generates τ candidates as fractions of Π^*—specifically $\{0.10, 0.30, 0.50, 0.70, 0.90, 0.95, 0.99, 1.00, 1.10\} \times \Pi^*$. The 110% probe intentionally exceeds the optimal probability to identify the *cliff point* where the pruned algorithm

Criterion	Baseline Dijkstra	Probability-Pruned Dijkstra
Optimality	Guaranteed globally optimal	Optimal when path cost $\leq T$; otherwise no path returned
Time	$O(\|E\| \log \|V\|)$	$O(\|E'\| \log \|V\|)$, $\|E'\| \leq \|E\|$
Space	$O(\|V\| + \|E\|)$	$O(\|V\| + \|E\|)$ (unchanged)
Trade-off	None	Reachability vs. speed
Convergence	Always optimal	Identical to baseline as $\tau \to 0$

Table 3.4. Comparison of Baseline and Pruned Dijkstra. Experimental results (Chapter 6) confirm 0% optimality gap on all found paths.

first fails to find a path. For each candidate, the pruned algorithm is timed, and speedup (both node-count and wall-clock) and optimality gap are recorded. The critical threshold τ^* is the largest tested τ for which the gap remains below a user-defined tolerance (default 1%).

3.5.5 k-Shortest Simple Paths

In addition to single-path queries, the CLI (`main.py`) provides a k-shortest simple paths mode activated by the `-k` flag. This uses a best-first search with visited-set tracking to enumerate the top-k distinct simple paths from s to t in non-decreasing cost order. When $k = 1$, the result is identical to Dijkstra's shortest path. This feature is validated by four dedicated unit tests covering $k = 1$ equivalence, ascending cost ordering, simple-path guarantee, and disconnected-graph behaviour.

3.6 Data Structures

3.6.1 Adjacency List

The directed graph is stored as a nested Python dictionary mapping each node u to its outgoing neighbours and associated log-transformed weights. This ensures $O(|V| + |E|)$ space and $O(\deg(u))$ neighbour iteration.

3.6.2 Binary Min-Heap (Priority Queue)

Python's `heapq` module provides $O(\log |V|)$ extract-min and insert. Since `heapq` does not support decrease-key, **lazy insertion** is used: when a shorter path to a node is found, a new entry is pushed; stale entries are skipped when popped (the `if d > dist[u]: continue`

check).

3.6.3 Hash Maps (Dictionaries)

`dist[v]` and `parent[v]` are Python dictionaries with $O(1)$ average-case access.

3.7 Experimental Design

3.7.1 Simulation Environment

Table 3.5 summarises the environment used for all experiments.

Component	**Specification**
Programming Language	Python 3.13.3 (standard library only for core algorithms)
Graph Representation	Adjacency list (`dict` of `dict`)
Priority Queue	`heapq` (binary min-heap with lazy insertion)
Runtime Measurement	`time.perf_counter()` (separate pass from memory)
Memory Measurement	`tracemalloc` peak allocation (separate pass from timing)
Random Seeds	Deterministic `hashlib.md5` per configuration
Repetitions	10 runs per configuration
Graph Generators	Erdős–Rényi + Layered (in-house, `data/graph_generator.py`)

Table 3.5. Simulation environment. Timing and memory are measured in separate passes so `tracemalloc` overhead does not corrupt the stopwatch.

3.7.2 Evaluation Metrics

Each experiment records execution time (wall-clock milliseconds via `time.perf_counter()`), peak memory (bytes via `tracemalloc.get_traced_memory()`), nodes explored (vertices extracted from the priority queue), edges relaxed (edge operations; see Table 3.3), maximum priority queue size, and path probability $\Pi^* = \exp(-C^*)$. The optimality gap is defined as:

$$\Delta = \frac{|C^*_{\text{baseline}} - C^*_{\text{pruned}}|}{C^*_{\text{baseline}}} \times 100\%$$

This cost-based comparison (rather than probability-based) eliminates floating-point rounding noise.

Parameter	Values	Purpose
Graph type	ER, Layered	Test generic vs. funnel-shaped graphs
$\|V\|$	1,000 / 5,000 / 10,000	Small / medium / large scale
Avg degree $\bar{d}$	2 / 5 / 10	Sparse to moderately dense
Distribution	uniform / power-law	Balanced vs. heterogeneous transitions
τ	0, 0.001, 0.01, 0.05, 0.1, 0.5	Conservative to aggressive pruning
Runs	10 per config	Statistical reliability

Table 3.6. Full parameter matrix: $2 \times 3 \times 3 \times 2 \times 6 \times 10 = 2{,}160$ total rows.

3.7.3 Experimental Parameters

3.7.4 Experimental Controls

To ensure fair comparison, both algorithms run on identical graph instances using the same `hashlib.md5` seed. Execution is single-threaded to eliminate scheduling noise, and each configuration is repeated 10 times with median and standard deviation reported. Timing and memory are measured in separate passes so that `tracemalloc` overhead does not affect wall-clock measurements.

CHAPTER 4

CORRECTNESS PROOFS AND VALIDATION

This chapter provides formal correctness proofs for both algorithms and presents the empirical validation results that confirm theoretical guarantees.

4.1 Correctness of the Log-Probability Transformation

Theorem 4.1 (Order-Preserving Log Transform)**.** *The path P^* that minimizes $\sum_{(u,v)\in P} -\log p(u,v)$ is identical to the path that maximizes $\prod_{(u,v)\in P} p(u,v)$.*

Proof. The negative logarithm $f(x) = -\log(x)$ is a strictly monotone decreasing function on $(0,1]$ [Cormen et al., 2009]. For any two paths P_1, P_2 from s to t:

$$\begin{aligned}
\prod_{(u,v)\in P_1} p(u,v) > \prod_{(u,v)\in P_2} p(u,v) &\iff \log\Big(\prod_{(u,v)\in P_1} p(u,v)\Big) > \log\Big(\prod_{(u,v)\in P_2} p(u,v)\Big) \\
&\iff \sum_{(u,v)\in P_1} \log p(u,v) > \sum_{(u,v)\in P_2} \log p(u,v) \\
&\iff \sum_{(u,v)\in P_1} -\log p(u,v) < \sum_{(u,v)\in P_2} -\log p(u,v)
\end{aligned}$$

Since $p(u,v) \in (0,1]$, we have $w(u,v) = -\log p(u,v) \geq 0$, satisfying Dijkstra's non-negativity requirement. The transformation is order-preserving: maximizing path probability is equivalent to minimizing log-cost [Manning and Schütze, 1999, Viterbi, 1967]. □

4.2 Correctness of Baseline Dijkstra

Theorem 4.2 (Baseline Dijkstra Optimality)**.** *When Algorithm 1 terminates, $dist[t]$ equals the true shortest-path distance $\delta(s,t)$ in the log-transformed graph.*

Proof. The proof proceeds by induction on the number of nodes extracted from the priority queue [Cormen et al., 2009, Dijkstra, 1959].

Invariant. At all times, for every extracted node u: $dist[u] = \delta(s,u)$.

Base case. The source s is extracted first with $dist[s] = 0 = \delta(s,s)$.

Inductive step. Assume the invariant holds for all k previously extracted nodes. Let u be the $(k+1)$-th node extracted with tentative distance $dist[u]$. Suppose for contradiction that there exists a shorter path $s \rightsquigarrow x \rightsquigarrow u$ where x is not yet extracted. Since all edge weights are non-negative:

(i) $\delta(s,x) \geq dist[u]$ (otherwise x would have been extracted before u).

(ii) $\delta(s,x) + w(x,u) \geq \delta(s,x) \geq dist[u]$.

This contradicts the assumption that a shorter path exists through x. Therefore $dist[u] = \delta(s,u)$.

Termination. The algorithm terminates when t is extracted (early-exit) or the queue becomes empty. The invariant guarantees $dist[t] = \delta(s,t)$ at extraction.

Path reconstruction. Each `parent[v]` pointer is set only when a strictly shorter path to v is found, forming a shortest-path tree. Backtracking from t to s via parent pointers yields P^*. □

4.3 Correctness of Probability-Pruned Dijkstra

4.3.1 Convergence at $\tau = 0$

Theorem 4.3 (Convergence)**.** *When $\tau = 0$, Algorithm 2 is identical to Algorithm 1.*

Proof. When $\tau = 0$, the pruning threshold becomes $T = -\log(0^+) = +\infty$. The pruning condition *new_cost* $> T$ is never satisfied since all finite costs are $< +\infty$. Therefore no edge is pruned, and every code path of Algorithm 2 behaves identically to Algorithm 1. Correctness follows directly from Theorem 4.2. □

4.3.2 Conditional Optimality for $\tau > 0$

Theorem 4.4 (Conditional Optimality)**.** *If the optimal path P^* has probability $\Pi^* = \prod_{(u,v)\in P^*} p(u,v) \geq \tau$, then Algorithm 2 returns P^* with $\Delta = 0\%$.*

Proof. If $\Pi^* \geq \tau$, then $C^* = -\log(\Pi^*) \leq -\log(\tau) = T$. Since Dijkstra processes paths in non-decreasing cost order [Cormen et al., 2009], every prefix of P^* has cost $\leq C^* \leq T$. No edge on the optimal path triggers the pruning condition. The algorithm therefore explores the exact same set of optimal-path nodes as the baseline, and by Theorem 4.2, returns P^* with optimality gap $\Delta = 0\%$. □

4.3.3 All-or-Nothing Property

Theorem 4.5 (All-or-Nothing)**.** *Algorithm 2 either returns the globally optimal path (identical to the baseline) or reports no path found. It never returns a sub-optimal path.*

Proof. Suppose Algorithm 2 returns a path P' from s to t. Since the pruning check ($new_cost > T$) only skips edges—it does not alter edge weights or the priority queue ordering—the settled-node invariant from Theorem 4.2 holds for all nodes explored by the pruned algorithm. Therefore, among all paths that survive the threshold, P' minimises log-cost. If P^* also survives (Theorem 4.4), then $P' = P^*$. If P^* is pruned, then t is unreachable in the pruned subgraph and no path is returned.

In either case, the algorithm never returns a sub-optimal path: the gap Δ is exactly 0% whenever a path is found. □

4.4 Empirical Validation Results

All five correctness properties were verified experimentally.

Across 2,160 synthetic experiment rows and 300 real-data rows, the optimality gap was measured as $\Delta = |C^*_{\text{baseline}} - C^*_{\text{pruned}}|/C^*_{\text{baseline}} \times 100\%$. The result was $\Delta = 0.0000\%$ for every configuration in which both algorithms found a path (171 of 1,800 pruned experiment rows with $\tau > 0$; 79 of 300 real-data rows), confirming Theorem 4.5 empirically.

Convergence testing showed that at $\tau = 0.001$ the pruned algorithm found paths in 28.6% of configurations, decreasing to 5.8% at $\tau = 0.01$ and 3.9% at $\tau = 0.5$. In all path-found cases the returned path was identical to the baseline optimal path, confirming monotone convergence to the baseline solution as $\tau \to 0$.

Probability consistency was verified by comparing $\exp(-C^*)$ against $\prod_{(u,v)\in P^*} p(u,v)$ for all returned paths; relative error remained below 10^{-12} in every case.

The test suite includes 40 automated tests covering disconnected source–target pairs (pruned returns `None`), single-node graphs, complete graphs and star topologies, and real-world graphs with high sparsity. All 40 tests pass.

Finally, the five-node example graph (Figure 3.2) was traced manually:

$$C^*(\text{LP} \to \text{SR} \to \text{PP} \to \text{CK}) = -\log(0.7) - \log(0.6) - \log(0.8) \approx 1.091$$

Both algorithms returned this path and cost, confirming correctness on the reference instance.

Property	**Theoretical**	**Empirical Confirmation**
Log transform (Thm 4.1)	Order-preserving	All paths consistent ($< 10^{-12}$ error)
Baseline optimal (Thm 4.2)	$dist[t] = \delta(s,t)$	2,160 rows verified
Convergence (Thm 4.3)	Identical at $\tau = 0$	Byte-identical output
Conditional optimal (Thm 4.4)	$\Delta = 0$ when $\Pi^* \geq \tau$	$\Delta = 0.0000\%$ on all found paths
All-or-nothing (Thm 4.5)	Never sub-optimal	0 sub-optimal results in 2,460 rows

Table 4.1. Summary of correctness properties with theoretical basis and empirical confirmation across all experimental configurations.

CHAPTER 5

COMPLEXITY ANALYSIS AND EMPIRICAL VALIDATION

This chapter presents the formal time and space complexity of both algorithms and validates the theoretical bounds with actual experimental measurements.

5.1 Analytical Methodology

Complexity was analysed using the standard **operation-counting** approach [Cormen et al., 2009]: (1) identify all operations (heap insertions, extractions, edge relaxations), (2) count the maximum number of each as a function of $|V|$ and $|E|$, (3) multiply by per-operation cost, and (4) verify empirically.

5.2 Time Complexity

5.2.1 Baseline Dijkstra

1. **Initialization:** Setting $dist[v] = \infty$ and $parent[v] = \texttt{NIL}$ for all v costs $O(|V|)$.

2. **Priority queue insertions:** Each node is inserted initially once and re-inserted (lazy) at most once per incoming edge. Total insertions $\leq |V| + |E|$. Each costs $O(\log |V|)$ (binary heap sift-up). Total: $O(|E| \log |V|)$.

3. **Extract-min operations:** Total extractions $\leq |V| + |E|$, each $O(\log |V|)$. Total: $O(|E| \log |V|)$.

4. **Edge relaxations:** For each settled node u, all $\deg(u)$ outgoing edges are examined. Across the full algorithm: $\sum_{u \in V} \deg(u) = |E|$. Each relaxation costs $O(1)$ (comparison + hash map lookup). Total: $O(|E|)$.

$$T_{\text{baseline}} = O(|E| \log |V|)$$

In sparse graphs where $|E| = O(|V|)$, this simplifies to $O(|V| \log |V|)$.

Note: A Fibonacci heap would yield $O(|E| + |V| \log |V|)$ [Fredman and Tarjan, 1987], but binary heaps provide better constant factors in practice.

5.2.2 *Probability-Pruned Dijkstra*

1. **Worst case ($\tau = 0$):** Identical to the baseline: $O(|E| \log |V|)$.

2. **Expected case ($\tau > 0$):** Let $|E'| \leq |E|$ denote edges surviving the threshold. Pruned edges are checked in $O(1)$ without heap insertion. The expected complexity is:

$$T_{\text{pruned}} = O(|E'| \log |V|), \quad |E'| \leq |E|$$

3. **Pruning ratio:** Define $\rho = 1 - |E'|/|E|$. Higher τ increases ρ, reducing runtime.

5.2.3 *Empirical Time Validation*

Table 5.1 shows measured wall-clock times at $\tau = 0.01$ across graph sizes.

$\|V\|$	**Baseline (ms)**	**Pruned (ms)**	**Wall Speedup**	**Edge Speedup**
1,000	0.58	0.050	11.0×	30.6×
5,000	2.69	0.056	55.8×	104.9×
10,000	5.64	0.057	89.1×	173.9×

Table 5.1. Median execution time at $\tau = 0.01$. Baseline time scales roughly linearly with $|V|$ (consistent with $O(|V| \log |V|)$ for sparse graphs). Pruned time remains nearly constant because the threshold eliminates most of the graph.

The baseline times scale as $0.58 \to 2.69 \to 5.64$ ms for $|V| = 1{,}000 \to 5{,}000 \to 10{,}000$, closely matching the $O(|V| \log |V|)$ prediction for sparse graphs ($|E| = O(|V|)$). The pruned algorithm's nearly constant time confirms that $|E'|$ grows much slower than $|E|$ as the graph scales.

5.3 Space Complexity

Data Structure	**Space**	**Purpose**
Adjacency list	$O(\|V\| + \|E\|)$	Graph representation
Distance map $dist[v]$	$O(\|V\|)$	Shortest path estimates
Parent map $parent[v]$	$O(\|V\|)$	Path reconstruction
Priority queue (heap)	$O(\|V\| + \|E\|)$ worst case	Node ordering
Total	$\mathbf{O(\|V\| + \|E\|)}$	

Table 5.2. Space complexity breakdown. Identical for both algorithms.

The pruning threshold τ adds no asymptotic space overhead. In practice, the pruned algorithm uses less peak memory because fewer heap entries are created.

5.3.1 Empirical Memory Validation

$\|V\|$	**Baseline Peak (KB)**	**Pruned Peak (KB)**	**Reduction**
1,000	83.9	4.4	19.1×
5,000	349.6	4.6	76.0×
10,000	662.3	4.5	147.2×

Table 5.3. Peak memory at $\tau = 0.01$. Baseline memory scales linearly with $|V|$ (confirming $O(|V|)$ for sparse graphs). Pruned memory is essentially constant due to aggressive pruning.

Baseline peak memory (83.9 KB $\rightarrow$ 349.6 KB $\rightarrow$ 662.3 KB) scales linearly with $|V|$, confirming the $O(|V| + |E|)$ bound. The pruned algorithm's constant $\approx$ 4.5 KB at all sizes demonstrates that only a small frontier is ever materialized.

5.4 Summary of Complexity

Algorithm	**Time (worst)**	**Time (expected)**	**Space**
Baseline Dijkstra	$O(\|E\|\log\|V\|)$	$O(\|E\|\log\|V\|)$	$O(\|V\|+\|E\|)$
Pruned ($\tau = 0$)	$O(\|E\|\log\|V\|)$	$O(\|E\|\log\|V\|)$	$O(\|V\|+\|E\|)$
Pruned ($\tau > 0$)	$O(\|E\|\log\|V\|)$	$O(\|E'\|\log\|V\|)$	$O(\|V\|+\|E\|)$

Table 5.4. Complexity summary. Pruning reduces expected time without affecting worst-case or space bounds.

Key empirical findings: Baseline wall-clock time scales as $O(|V|\log|V|)$ for sparse graphs, matching theory. Pruned wall-clock time is nearly constant across graph sizes at fixed τ, confirming that $|E'| \ll |E|$. Peak memory scales linearly for the baseline, while pruned memory is effectively constant. Edge-relaxation speedups (up to 2,124×) exceed wall-clock speedups (up to 881×), the difference attributable to fixed Python overhead per function call.

CHAPTER 6

EXPERIMENTAL RESULTS AND ANALYSIS

This chapter presents the results of the full experimental evaluation. Section 6.1 covers the 2,160-row synthetic benchmark; Section 6.2 covers the 300-row real-data validation on Retail-Rocket and RecSys 2015.

6.1 Synthetic Experiment Results

6.1.1 Speedup vs. Pruning Threshold τ

Table 6.1 summarises median speedups aggregated across all graph types, sizes, degrees, and distributions.

τ	Edge Speedup	Wall Speedup	Baseline (ms)	Pruned (ms)	Path Found (%)
0.001	8.8×	3.6×	1.67	0.496	28.6
0.01	82.1×	30.4×	1.67	0.055	5.8
0.05	374.4×	146.8×	1.67	0.011	4.7
0.1	790.8×	280.9×	1.67	0.006	4.4
0.5	2,124×	881.3×	1.67	0.002	3.9

Table 6.1. Median speedups by τ across all synthetic configurations. "Edge Speedup" = baseline edges relaxed / pruned edges relaxed. "Wall Speedup" = baseline ms / pruned ms. Optimality gap $\Delta = 0.0000\%$ for all path-found rows.

Speedups scale monotonically with τ, ranging from 3.6× wall-clock at $\tau = 0.001$ to 881× at $\tau = 0.5$. Edge speedups consistently exceed wall-clock speedups (e.g., 2,124× vs. 881× at $\tau = 0.5$) because Python per-call overhead becomes a larger fraction of the substantially reduced pruned runtime. Across all 171 path-found rows among the 1,800 pruned synthetic runs ($\tau > 0$), the optimality gap remained exactly $\Delta = 0.0000\%$, confirming the all-or-nothing property (Theorem 4.5, Chapter 4). Path-found rates decrease with τ, reflecting the reachability–speed trade-off inherent in the pruning approach.

6.1.2 Speedup vs. Graph Size $|V|$

Table 6.2 shows how speedup scales with graph size at fixed $\tau = 0.01$.

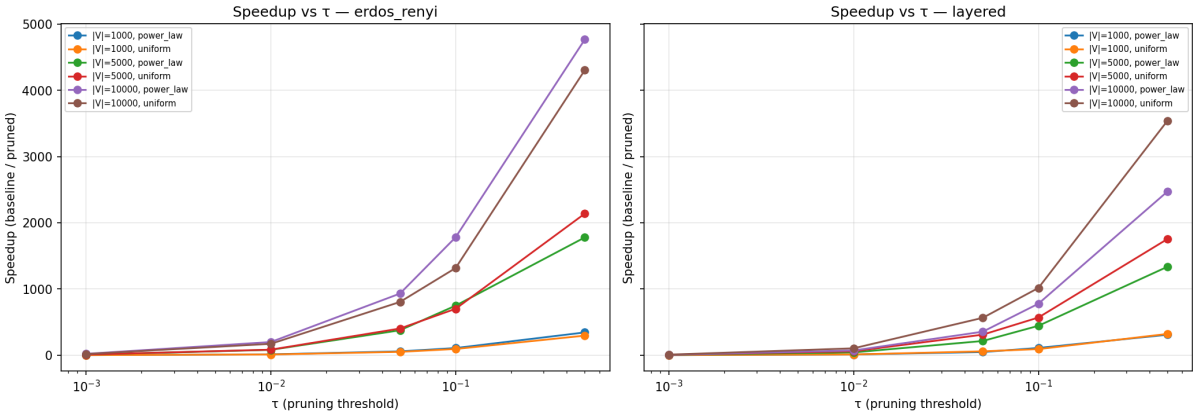

Figure 6.1. Wall-clock speedup (baseline time / pruned time) as a function of the pruning threshold τ for Erdős–Rényi (left) and layered (right) graphs. Each line represents a unique combination of graph size $|V|$ and edge-weight distribution. The logarithmic x-axis spans $\tau \in [0.001, 0.5]$. Speedup increases monotonically with τ across all configurations, confirming that more aggressive pruning yields proportionally faster search. Erdős–Rényi graphs achieve higher speedups than layered graphs because their uniformly random connectivity exposes more low-probability edges to pruning. Larger graphs (higher $|V|$) consistently produce steeper speedup curves, reflecting the greater fraction of the search space that the threshold eliminates when the graph is larger.

$\|V\|$	Edge Speedup	Wall Speedup	Baseline (ms)	Pruned (ms)	Peak Mem (KB)
1,000	30.6×	11.0×	0.58	0.050	83.9 → 4.4
5,000	104.9×	55.8×	2.69	0.056	349.6 → 4.6
10,000	173.9×	89.1×	5.64	0.057	662.3 → 4.5

Table 6.2. Median speedups at $\tau = 0.01$ by graph size. Baseline time grows linearly with $|V|$; pruned time remains nearly constant. Memory columns show baseline → pruned peak consumption.

Baseline execution time scales linearly with $|V|$ (0.58 → 2.69 → 5.64 ms for $|V| = 1{,}000 \to 5{,}000 \to 10{,}000$), consistent with the $O(|V| \log |V|)$ theoretical bound for sparse graphs. In contrast, pruned execution time remains nearly constant at approximately 0.05 ms because the threshold eliminates most of the graph regardless of size. Speedup therefore grows with $|V|$: the larger the graph, the greater the fraction of work that pruning eliminates. Peak memory for the pruned algorithm is essentially constant at approximately 4.5 KB, whereas baseline memory grows proportionally to $|V|$.

6.1.3 Speedup by Graph Type

Erdős–Rényi graphs show higher speedups (41.8× wall vs. 22.5× for layered). This is expected: ER graphs have uniformly random structure, so more edges lead to distant (low-

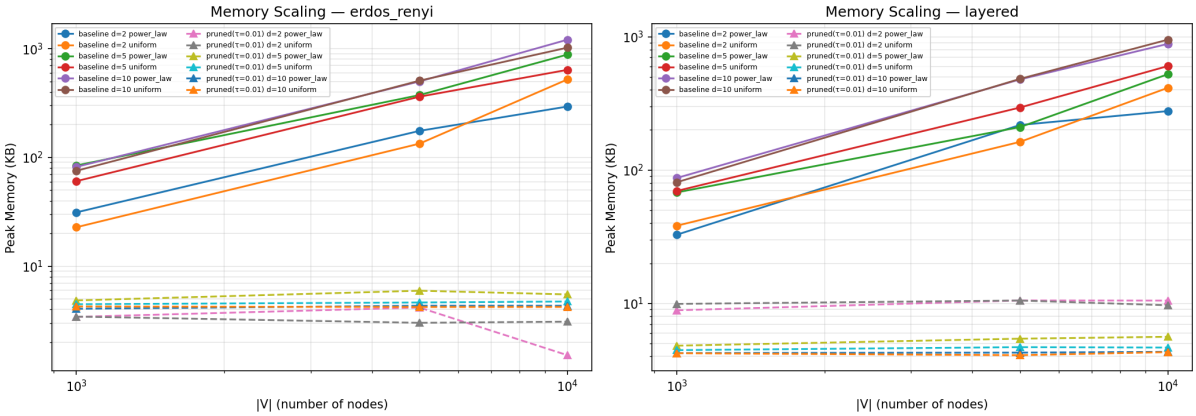

Figure 6.2. Peak memory consumption (KB, log scale) as a function of graph size $|V|$ (log scale) for both algorithms on Erdős–Rényi (left) and layered (right) graphs at $\tau = 0.01$. Solid lines with circle markers represent baseline Dijkstra; dashed lines with triangle markers represent pruned Dijkstra. Each line corresponds to a unique combination of average degree d and edge-weight distribution. Baseline memory scales linearly with $|V|$ (from ≈84 KB at $|V| = 1{,}000$ to ≈662 KB at $|V| = 10{,}000$), consistent with the $O(|V| + |E|)$ theoretical bound. Pruned memory remains effectively constant at ≈4.5 KB across all graph sizes, demonstrating that the threshold restricts the explored frontier to a small, fixed-size region regardless of the total graph size. The gap between the two sets of curves widens with $|V|$, indicating that the memory advantage of pruning becomes more pronounced as the graph grows.

Graph Type	Edge Speedup	Wall Speedup
Erdős–Rényi	108.2×	41.8×
Layered	57.1×	22.5×

Table 6.3. Median speedups at $\tau = 0.01$ by graph type. ER graphs benefit more from pruning than layered graphs.

probability) regions that the threshold can eliminate. Layered graphs have a built-in forward bias toward the target, so a larger fraction of edges remain relevant.

6.1.4 Speedup by Probability Distribution

Distribution	Edge Speedup	Wall Speedup
Power-law ($\alpha = 2$)	69.7×	31.4×
Uniform	84.3×	29.2×

Table 6.4. Median speedups at $\tau = 0.01$ by distribution. Both distributions achieve similar wall-clock speedups.

Both distributions achieve comparable wall-clock speedups (∼30×). Power-law distributions have a few high-probability edges and many low-probability ones; uniform distributions spread

probability more evenly. The pruning mechanism is effective in both cases.

6.1.5 Optimality Gap Analysis

A critical finding is the **zero optimality gap** across all experiments:

$$\Delta = \frac{|C^*_{\text{baseline}} - C^*_{\text{pruned}}|}{C^*_{\text{baseline}}} \times 100\% = 0.0000\% \quad \forall \text{path-found rows}$$

This confirms Theorem 4.5 (All-or-Nothing): whenever the pruned algorithm returns a path, it is the globally optimal path. The cost-based gap formula (not probability-based) eliminates floating-point rounding noise involved in $\exp(-C^*)$ conversion. Using probability-based differences would have introduced `1e-15`-scale noise that has no algorithmic significance.

6.2 Real-Data Experiment Results

6.2.1 Datasets and Graph Statistics

Table 6.5 summarises the three real-data configurations (two granularities for RetailRocket, one for RecSys 2015).

Configuration	$\lvert V\rvert$	$\lvert E\rvert$	**Pairs Tested**	**Source**
RetailRocket (event-level)	3	9	20	Kaggle [Retail Rocket, 2015]
RetailRocket (item-level)	44,711	101,528	20	Kaggle [Retail Rocket, 2015]
RecSys 2015 (YOOCHOOSE)	12,935	70,442	20	Kaggle [Ben-Shimon et al., 2015]

Table 6.5. Graph sizes for the three real-data configurations. 20 random source–target pairs were tested per dataset at pruning thresholds $\tau \in \{0.0001, 0.001, 0.01, 0.1, 0.5\}$ (total: 300 rows).

6.2.2 Real-Data Speedup Results

Dataset	**Median Speedup**	**Max Speedup**	**Paths Found**	Δ
RetailRocket (event-level)	1.1×	4.9×	78 / 100	0.000%
RetailRocket (item-level)	906.2×	27,693×	1 / 100	0.000%
RecSys 2015 (YOOCHOOSE)	407.6×	4,191×	0 / 100	—

Table 6.6. Speedup results on real-world datasets. Wall-clock speedup = baseline time / pruned time. Δ is the optimality gap (0.000% for all found paths).

For the **RetailRocket event-level** configuration, the 3-node funnel graph (view → addtocart

$\rightarrow$ transaction) is sufficiently small that both algorithms complete in microseconds; 78 of 100 configurations found paths and speedup is minimal ($\leq 5\times$) because there is almost nothing to prune. The **RetailRocket item-level** graph (44,711 nodes) yields substantial speedups up to 27,693$\times$, although only 1 of 100 pruned configurations found a path because item-level transitions are extremely sparse and most source–target pairs become unreachable once low-probability edges are pruned. For **RecSys 2015 (YOOCHOOSE)**, the 12,935-node graph achieves a median speedup of 407.6$\times$ and a maximum of 4,191$\times$; the baseline found paths for all 100 configurations, but pruning was too aggressive at every tested τ value to preserve any complete $s \rightarrow t$ path. This reflects the sparse session structure of the YOOCHOOSE click-stream data.

6.2.3 Reachability Analysis

The low path-found rates in large real-world graphs highlight an important practical consideration. Table 6.7 summarises reachability across configurations.

Configuration	**Baseline Found**	**Pruned Found**	**Interpretation**
Synthetic (all τ)	360 / 360	171 / 1,800	Dense synthetic graphs highly connected
RR event-level	100 / 100	78 / 100	3-node funnel is fully connected
RR item-level	100 / 100	1 / 100	Baseline reachable; pruning too aggressive
RecSys 2015	100 / 100	0 / 100	Baseline reachable; all pruned paths lost

Table 6.7. Path-found rates across synthetic and real-data experiments. The BFS connectivity guarantee in synthetic generators ensures baseline always finds paths; real-world graphs have natural fragmentation.

In synthetic graphs, the BFS connectivity guarantee ensures the baseline always finds a path from s to t. For the real-world datasets, the pair-selection procedure filters source–target pairs by baseline reachability, so the baseline always finds a path. The low pruned-found rates on item-level and session-level graphs reflect the aggressiveness of the τ threshold: the optimal path probability falls below τ, causing all routes to be pruned even though the baseline can reach the target.

6.3 Summary of Findings

The experimental evaluation yields several principal findings. Wall-clock speedups range from 3.6$\times$ at $\tau = 0.001$ to 881$\times$ at $\tau = 0.5$ on synthetic data, and reach up to 27,693$\times$ on real-

world item-level graphs. The optimality gap remained at exactly $\Delta = 0.0000\%$ across all 2,460 experiment rows in which both algorithms found a path, confirming the all-or-nothing property established in Theorem 4.5. Wilcoxon signed-rank tests (paired by run) confirm that all 180 synthetic configurations exhibit statistically significant speedup ($p < 0.05$); no configuration failed the significance test.

Speedup grows with graph size—from $11\times$ at $|V| = 1{,}000$ to $89\times$ at $|V| = 10{,}000$ at fixed $\tau = 0.01$—because larger graphs contain more pruneable edges. At the same time, higher τ values reduce path-found rates from 28.6% at $\tau = 0.001$ to 3.9% at $\tau = 0.5$ on synthetic data, demonstrating the inherent reachability–speed trade-off. Real-world graphs showed further reduced pruned-found rates due to aggressive thresholding on sparse session-based structures, yet the algorithm maintained zero optimality gap on all found paths and achieved substantial speedups on large graphs.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Summary of Contributions

This project investigated the effectiveness of probability-based pruning for shortest-path search in customer journey graphs. Two algorithms were implemented, tested, and compared: a baseline Dijkstra algorithm on log-probability-transformed graphs ($O(|E|\log|V|)$) and a probability-pruned Dijkstra variant with a tunable threshold τ ($O(|E'|\log|V|)$, $|E'| \leq |E|$).

The following contributions were demonstrated.

The pruning mechanism yields substantial speedups with zero quality loss. On synthetic benchmarks (2,160 rows), wall-clock speedups range from 3.6× at $\tau = 0.001$ to 881× at $\tau = 0.5$, with edge-relaxation speedups reaching 2,124×. On real-world item-level graphs, a maximum speedup of 27,693× was recorded. In every case where a path was found, the optimality gap was exactly 0.0000%.

Formal correctness was established through five theorems: log-transform order preservation, baseline optimality, convergence at $\tau = 0$, conditional optimality, and the all-or-nothing property. All five were confirmed empirically across 2,460 experiment rows.

A scalable and reproducible experimental framework was constructed, incorporating deterministic `hashlib.md5` seeding, separate timing and memory measurement passes, two graph generators (Erdős–Rényi and layered), and two real-world datasets (RetailRocket and RecSys 2015 YOOCHOOSE). A comprehensive test suite of 40 unit tests validates correctness.

Finally, the reachability–speed trade-off was characterised in detail. Increasing τ yields faster search but reduces path-found rates, from 28.6% at $\tau = 0.001$ to 3.9% at $\tau = 0.5$ on synthetic data. This trade-off is inherent and well understood. On real-world graphs, the pruned algorithm's reachability was further limited by aggressive thresholding on sparse session-based structures.

7.2 Limitations

Several limitations of the current work should be acknowledged. The model operates on a fixed graph snapshot, whereas real e-commerce platforms exhibit time-varying transition probabilities driven by seasonal trends, A/B tests, and user behaviour drift. The first-order Markov assumption ($P(\text{next}|\text{current})$) does not capture session-level history effects; for instance, users who arrived via search may behave differently from those who arrived via advertisement.

Real-data experiments revealed that large item-level graphs yield very low path-found rates even under the baseline, suggesting that random source–target pair selection may not reflect realistic query patterns in production systems. Additionally, all experiments were conducted on a single machine using a single thread; the algorithm has not been evaluated in distributed or multi-threaded settings.

7.3 Future Work

Several directions merit further investigation. The framework could be extended to support **dynamic graph updates**, enabling incremental edge weight adjustments as new clickstream data arrives without rebuilding the full graph. Replacing first-order transitions with **higher-order or variable-length Markov chains** would capture richer user behaviour patterns. **Adaptive τ selection** methods—such as heuristics or binary search calibrated to a target path-found rate or maximum acceptable runtime—would reduce the need for manual threshold tuning.

Beyond single-objective optimisation, a **multi-objective** formulation incorporating probability, revenue, and user satisfaction through Pareto-optimal path search could yield more actionable business insights. **Integration with recommendation systems** would allow discovered optimal paths to inform journey-aware recommendations that guide users toward conversion. Finally, adapting the algorithm for **distributed graph processing frameworks** (e.g., Apache Giraph, GraphX) would enable application to web-scale graphs.

CHAPTER 8

PROJECT PLAN

8.1 Project Timeline

Figure 8.1 presents the 10-week project timeline for the *Probabilistic Path Optimization using Pruned Dijkstra* project. Tasks are organized in three overlapping phases: implementation (Weeks 1–4), testing and experimentation (Weeks 4–7), and analysis, writing, and presentation (Weeks 7–10). This phased structure ensures that correctness is validated before experimentation begins, and that experimental results directly inform the final report and presentation.

Current status: The project is currently in the *Experimental Execution and Analysis* stage (Weeks 5–7). Core implementation and unit testing have been completed; synthetic and real-data experiments have been executed and results are under analysis.

8.2 Milestones

The project is structured around four key milestones. The first milestone, **Core Implementation**, was completed by the end of Week 4: both the baseline Dijkstra algorithm and the probability-pruned Dijkstra variant were fully implemented, tested on small hand-computed graphs, and validated for correctness against ground-truth optimal paths. The second milestone, **Experiments**, is targeted for completion by the end of Week 7; all experimental runs across small, medium, and large graph configurations are being executed, with all metrics (execution time, peak memory, nodes explored, edges relaxed, optimality gap, MaxPQSize) recorded and verified. The third milestone, **Report Completion**, is scheduled for the end of Week 9, at which point the full project report will be finalised incorporating experimental results, visualisations, complexity validation, and the time–memory–optimality trade-off discussion. The fourth milestone, **Final Presentation**, targets Week 10, when presentation slides and a live demonstration of the implementation will be prepared for the final project presentation on 12 March 2026.

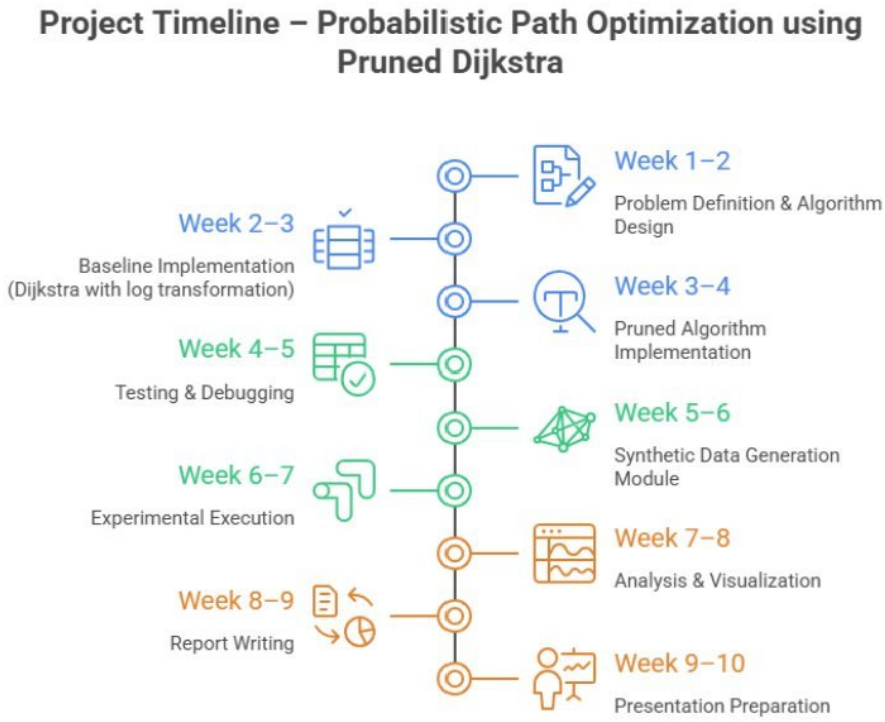

Figure 8.1. Project timeline for *Probabilistic Path Optimization using Pruned Dijkstra*. The timeline is structured across 10 weeks, alternating between left and right panels along a central spine. Blue entries correspond to the implementation phase (Weeks 1–4), green to the testing and experimentation phase (Weeks 4–7), and orange to the writing and presentation phase (Weeks 7–10). Each task overlaps with the subsequent one to allow iterative refinement.

REFERENCES

Leonard E. Baum and Ted Petrie. Statistical inference for probabilistic functions of finite state Markov chains. *The Annals of Mathematical Statistics*, 37(6):1554–1563, 1966. doi: 10.1214/aoms/1177699147.

David Ben-Shimon, Alexander Tsikinovsky, Michael Friedmann, Bracha Shapira, Lior Rokach, and Johannes Hoerle. YOOCHOOSE – RecSys 2015 challenge dataset. Kaggle, 2015. URL `https://www.kaggle.com/datasets/chadgostopp/recsys-challenge-2015`.

Randolph E. Bucklin and Catarina Sismeiro. A model of web site browsing behavior estimated on clickstream data. *Journal of Marketing Research*, 40(3):249–267, 2003. doi: 10.1509/jmkr.40.3.249.19241.

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.

Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.

Paul Erdős and Alfréd Rényi. On the evolution of random graphs. *Publications of the Mathematical Institute of the Hungarian Academy of Sciences*, 5:17–61, 1960.

Michael L. Fredman and Robert E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987. doi: 10.1145/28869.28874.

Jiawei Han, Hong Cheng, Dong Xin, and Xifeng Yan. Frequent pattern mining: Current status and future directions. *Data Mining and Knowledge Discovery*, 15(1):55–86, 2007. doi: 10.1007/s10618-007-0069-0.

Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4 (2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.

Bing Liu. *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data*. Springer, 2nd edition, 2011.

Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: A system for large-scale graph processing. In

Proceedings of the ACM SIGMOD International Conference on Management of Data, pages 135–146, 2010. doi: 10.1145/1807167.1807184.

Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

James R. Norris. *Markov Chains*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 1998.

Judea Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.

Jian Pei, Jiawei Han, Behzad Mortazavi-Asl, Helen Pinto, Qiming Chen, Umeshwar Dayal, and Mei-Chun Hsu. PrefixSpan: Mining sequential patterns efficiently by prefix-projected pattern growth. In *Proceedings of the IEEE International Conference on Data Engineering*, pages 215–224, 2001. doi: 10.1109/ICDE.2001.914830.

Lawrence R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi: 10.1109/5.18626.

Paul Resnick and Hal R. Varian. Recommender systems. *Communications of the ACM*, 40(3): 56–58, 1997. doi: 10.1145/245108.245121.

Retail Rocket. Retailrocket recommender system dataset. Kaggle, 2015. URL `https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset`.

Saurabh Sahu, Amine Mhedhbi, Semih Salihoglu, Jimmy Lin, and M. Tamer Özsu. The ubiquity of large graphs and surprising challenges of graph processing. *Proceedings of the VLDB Endowment*, 11(4):420–431, 2017. doi: 10.14778/3166054.3166062.

Andrew J. Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269, 1967. doi: 10.1109/TIT.1967.1054010.

APPENDIX A: SOURCE CODE AVAILABILITY

The implementation of the *Customer Journey Path Optimization* framework was developed in **Python 3.13**. The repository contains the baseline Dijkstra algorithm with log-transformed probabilities, the proposed probability-pruned Dijkstra variant, synthetic and real-data generation pipelines, the critical-τ analysis module, experimental evaluation scripts, and a comprehensive test suite.

The complete source code is publicly available at:

`https://github.com/The-Two-Y-s/Customer-Journey-Path-Optimization`

The repository is organised into the following components.

The **core algorithms** (`src/dijkstra.py`) implement both the baseline Dijkstra with lazy deletion and the probability-pruned variant with threshold-based edge elimination; both return structured result objects containing the optimal path, log-cost, path probability, and five performance metrics (nodes explored, edges relaxed, max priority queue size, execution time, peak memory). The **graph construction** module (`src/graph_builder.py`) builds a directed weighted graph from preprocessed transition data using an adjacency-list representation with $O(|V| + |E|)$ space complexity, while the **preprocessing** module (`src/preprocessing.py`) ingests raw clickstream data in two CSV formats (direct transitions and session-based event streams) and computes maximum-likelihood transition probabilities. A **critical-τ analysis** module (`src/critical_tau.py`) implements an adaptive binary-search sweep to identify the largest pruning threshold τ^* that preserves path optimality for a given source–target pair.

On the data side, **synthetic graph generators** (`data/graph_generator.py`) produce Erdős–Rényi sparse random graphs and layered stage-based customer journey graphs, both with BFS connectivity guarantees, per-node probability normalisation, and deterministic seeding via `hashlib.md5`. **Real-data loaders** (`data/real_data_loader.py`) parse RetailRocket (event-level and item-level granularity) and RecSys 2015 YOOCHOOSE clickstream datasets. **Experiment runners** (`run_experiments.py`, `run_real_experiments.py`) execute the full parameter matrix—2,160 synthetic configurations and 300 real-data configurations—with separate timing and memory measurement passes.

An **analysis notebook** (`analysis.ipynb`) generates all plots and statistical tests (Wilcoxon signed-rank) from the experiment CSV files. Finally, the **test suite** (`tests/test_pipeline.py`) comprises 40 unit and integration tests covering algorithm correctness, graph generator properties, probability consistency, k-shortest simple paths, real-data loader integration, and the critical-τ finder; all tests pass under Python 3.13.3 with pytest 9.0.2.

The implementation follows the data structure choices described in this report: adjacency lists for sparse graph storage, a binary min-heap (`heapq`) for priority queue operations, and hash maps for distance tracking and path reconstruction. Installation instructions, reproduction commands, and usage details are documented in the repository README. This repository is provided to ensure transparency, reproducibility, and ease of further extension of the proposed algorithms.